

INTERVIEW MASTERY SERIES

Microservices

Interview Questions & Answers — The Complete 2026 Guide. From fundamentals to advanced distributed systems, with Spring Boot & Spring Cloud examples, real architecture diagrams, and the new AI-microservices patterns.

Updated for Spring Boot 4 · Spring Cloud 2025.1 · Spring AI

📐 50+ Questions | 🖼️ Architecture Diagrams | ⌨️ Real Code | 🎯 Scenarios & Trade-offs

TechTribers

Built by developers, for developers

★★★★★

Trusted by 26,200+ developers

www.techtriber.com

WHAT'S INSIDE

Table of Contents

Nine sections, basic to advanced — each question includes an explanation, code, a diagram where it helps, a real interview scenario, and the trade-offs interviewers actually probe for.

01 **Fundamentals** — Monolith vs microservices, DDD, bounded contexts, when NOT to use them — 8 Qs

02 **Communication** — Sync vs async, REST/gRPC/messaging, API versioning, idempotency — 7 Qs

03 **Service Discovery & Routing** — Eureka, lb:// scheme, Spring Cloud Gateway 5.0, BFF — 6 Qs

04 **Resilience** — Circuit breaker, retry, bulkhead, rate limiting, Resilience4j — 6 Qs

05 **Data & Transactions** — DB-per-service, Saga, Outbox, CQRS, event sourcing — 7 Qs

06 **Event-Driven & Messaging** — Kafka vs RabbitMQ, Spring Cloud Stream, delivery guarantees — 5 Qs

07 **Observability & Security** — Tracing, the three pillars, OAuth2/OIDC, zero-trust, mTLS — 6 Qs

08 **AI Microservices** 🤖 — Spring AI, RAG-as-a-service, MCP, agentic workflows, LLM gateway — 6 Qs

09 **System Design Questions** — Checkout, order processing with saga, AI assistant service — 4 Qs

CURRENT AS OF 2026

2026 Stack Snapshot

The Spring ecosystem had a major version jump in late 2025. This guide reflects the current stack — so your answers stay credible in senior interviews.

FRAMEWORK



Spring Boot 4.0

current 4.0.6 · on Spring Framework 7

Shipped Nov 2025. Jakarta EE 11, Jackson 3 default, JSpecify null-safety, modularized into 70+ jars, first-class Java 25 support (Java 17 minimum). Ships with Spring Security 7.

CLOUD

Spring Cloud 2025.1

"Oakwood" · all modules 5.0.x

The release train aligned with Spring Boot 4. Gateway 5.0 adds API-versioning predicate; Commons 5.0 adds circuit breaker for Interface Clients and the lb:// scheme. The old starter-parent artifact was removed.

AI



Spring AI

1.1 GA · 2.0 at milestone

1.0 GA (May 2025); 1.1 GA (Nov 2025) added Model Context Protocol (MCP) integration. 2.0 targets Spring Framework 7 / Boot 4, GA expected mid-2026. Powers the AI-microservices patterns in Section 08.

RESILIENCE



Resilience4j + Declarative

standard circuit breaker

Resilience4j remains the standard library. Spring Cloud Commons 5.0 now offers declarative resilience on annotated Spring Interface Clients — circuit breaking without manual wiring.

SECTION 01

Fundamentals

What microservices really are, how they differ from a monolith, and the design thinking — bounded contexts, granularity, and knowing when NOT to use them.



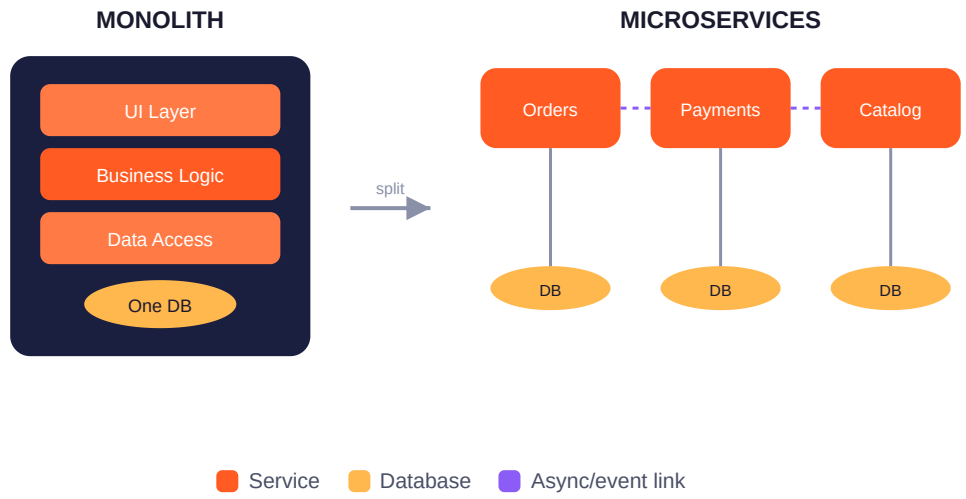
Q1 What are microservices, and how do they differ from a monolith?

🟢 Beginner | Fundamentals

💡 Explanation

A **monolith** packages all functionality — UI, business logic, data access — into a single deployable unit that scales as one block and shares one database. A **microservices** architecture decomposes the system into small, independently deployable services, each owning a single business capability and (ideally) its own data store. Services communicate over the network via APIs or messaging. The key shift isn't technical — it's organizational. Each service can be developed, deployed, scaled, and even rewritten by an independent team without coordinating a system-wide release.

📐 Diagram



🎯 Scenario

"Our monolith takes 40 minutes to build and one risky deploy ships everything at once." — Microservices let teams deploy independently, but the interviewer's real follow-up is: *"What new problems did you just buy?"* (network latency, distributed transactions, operational complexity). Naming those proves seniority.

⚠️ Trade-off / Gotcha

Microservices trade **in-process simplicity for distributed complexity**. You gain independent scaling and deployment; you pay with network calls that can fail, eventual consistency, and far harder debugging. Don't claim microservices are "better" — say they're a fit for large teams and independently-scaling domains, and overkill for small teams or simple apps.

Q2 When should you NOT use microservices?

🟡 Intermediate

Fundamentals

💡 Explanation

Microservices are an answer to *organizational and scaling* pressure, not a default starting point. Avoid them when:

- **Small team / early product** — you'll spend more time on infrastructure than features. A well-structured monolith ships faster.
- **Unclear domain boundaries** — if you don't yet understand your bounded contexts, you'll draw service lines in the wrong places and pay a brutal cost to redraw them.
- **Tight, chatty coupling** — if two "services" must always change together and call each other constantly, they're one service wearing a costume.
- **No operational maturity** — without CI/CD, observability, and on-call discipline, distributed systems become un-debuggable.

🎯 Scenario

A 4-person startup asks if they should "start with microservices to be scalable." The senior answer: start with a **modular monolith** — clean module boundaries inside one deployable. You get most of the design discipline with none of the distributed-systems tax, and you can carve out services later when a specific module actually needs independent scaling.

⚠️ Trade-off / Gotcha

The classic mistake is the **distributed monolith**: services that are deployed separately but so tightly coupled they must be released together. You get every downside of distribution and none of the independence. It's worse than the monolith you started with.

Q3 What is a bounded context, and why does it matter for service boundaries?

🟡 Intermediate | DDD

💡 Explanation

A **bounded context** (from Domain-Driven Design) is a boundary within which a domain model and its language are consistent. The word "Customer" might mean something different in the *Billing* context than in the *Support* context — bounded contexts make that explicit and keep each model coherent.

They matter because **a good microservice maps to one bounded context**. Draw service boundaries along business capabilities (and their language), not along technical layers. A service split by bounded context changes for one reason; a service split by technical layer changes every time any feature touches that layer.

⌨️ Heuristic

boundary-test.txt

```
// Good boundary — changes for ONE business reason:
OrderService      // everything about placing/tracking an order
PaymentService    // everything about charging money
InventoryService  // everything about stock levels

// Bad boundary — split by technical layer, changes constantly:
ValidationService // validates for orders, payments, inventory...
DatabaseService   // every feature has to touch it
```

⚠️ Trade-off / Gotcha

Don't over-decompose. "One service per database table" or "one service per entity" is a red flag — it creates dozens of chatty services that can't do anything useful alone. A bounded context is usually **several entities** that form one cohesive capability.

Q4 How do you decide service granularity — how small is too small?

🔴 Advanced | Design

💡 Explanation

Granularity is a balance between **independence** and **coordination cost**. Too coarse and you're back to a monolith; too fine and a single user action fans out into a dozen network hops, each a new failure point. Useful signals that a service is the right size:

- It owns its data and can be reasoned about in isolation.
- A team can hold its entire responsibility in their heads.
- It rarely needs synchronous chains through 3+ other services to do its job.
- It can be deployed without a coordinated release of its neighbours.

🎯 Scenario

"Every checkout calls 9 services synchronously and p99 latency is 4 seconds." That's a granularity smell — either the boundaries are wrong or the calls should be async/event-driven. The fix is often to **merge over-split services** or replace synchronous chains with events, not to add more services.

⚠️ Trade-off / Gotcha

Latency is additive and availability is multiplicative across a synchronous call chain. Five services each at 99.9% availability, called in sequence, give you ~99.5% — roughly **4× more downtime** than a single service. Granularity decisions are availability decisions.

Q5 What is Conway's Law and why do microservices teams care about it?

🟡 Intermediate

Org Design

💡 Explanation

Conway's Law: organizations design systems that mirror their own communication structure. If you have four teams, you'll tend to produce four major components — whether you planned to or not. Microservices teams use this deliberately (the "Inverse Conway Maneuver"): structure your **teams** around the bounded contexts you want, and the architecture follows. Each service should be owned end-to-end by one team — "you build it, you run it." Cross-team services become coordination bottlenecks and erode the independence microservices are supposed to give you.

⚠️ Trade-off / Gotcha

If a single service needs three teams to change it, your architecture and your org chart are fighting. Either give it one owner or split it along the lines the teams actually work.

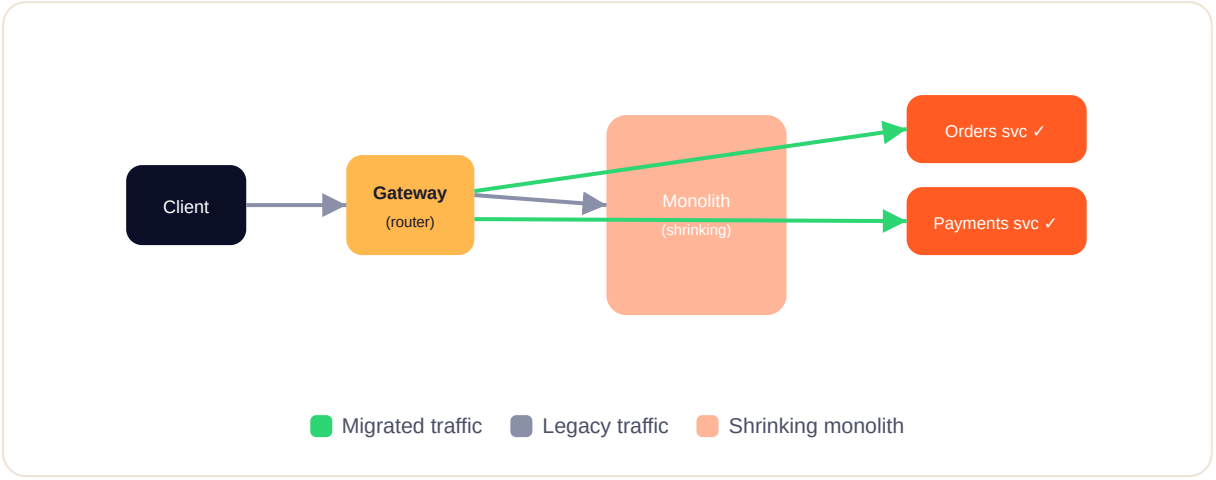
Q6 What is the strangler-fig pattern for migrating a monolith?

🔴 Advanced | Migration

💡 Explanation

Named after a vine that grows around a tree until it replaces it, the **strangler-fig** pattern migrates a monolith incrementally instead of in a risky "big bang" rewrite. You place a routing layer (often the API gateway) in front of the monolith, then peel off one capability at a time into a new service — routing that capability's traffic to the new service while everything else still hits the monolith. Over time the monolith shrinks to nothing.

📐 Diagram



⚠️ Trade-off / Gotcha

The hardest part is rarely the code — it's the **data**. While both systems run, the monolith and the new service may need the same data. You'll often need synchronization (CDC, events, or an anti-corruption layer) during the transition, and a clear plan for which system owns the write.

Q7 What are the fallacies of distributed computing, and why should every microservices engineer know them?

🟡 Intermediate

Fundamentals

💡 Explanation

A classic list of **false assumptions** developers make when moving from in-process calls to network calls. The big ones:

- **The network is reliable** — it isn't; calls drop, time out, and partition.
- **Latency is zero** — a method call is nanoseconds; a network hop is milliseconds, and they add up.
- **Bandwidth is infinite** — large payloads and chatty calls saturate it.
- **The network is secure** — assume it isn't; encrypt and authenticate.
- **Topology doesn't change** — instances come and go constantly in cloud/K8s.
- **Transport cost is zero** — serialization, TLS, and connection setup all cost.

Every microservices resilience pattern — timeouts, retries, circuit breakers — exists precisely because these assumptions are false.

⚠️ Trade-off / Gotcha

The most common rookie symptom: code that calls a remote service exactly like a local method — no timeout, no retry, no fallback. The first network blip takes the whole request down. If you remember only one fallacy, remember **the network is not reliable**.

Q8 Monolith, microservices, or modular monolith — how do you choose?

🔴 Advanced | Architecture

💡 Explanation

Think of it as a spectrum, chosen by team size, domain clarity, and scaling needs:

- **Monolith** — one deployable, one DB. Best for small teams, early products, simple domains. Fastest to ship.
- **Modular monolith** — one deployable, but enforced internal module boundaries (e.g. Spring Modulith). You get domain discipline and easy refactoring without distributed-systems cost. The best default for most growing teams.
- **Microservices** — many deployables, independent scaling/deploys. Worth it when you have multiple teams, clear bounded contexts, and parts of the system that genuinely need to scale or evolve independently.

🎯 Scenario

The senior move in an interview is to refuse the false binary. Don't say "microservices are modern, monoliths are legacy." Say: "I'd start as a modular monolith, keep boundaries clean, instrument everything, and extract a service only when a specific module proves it needs independent scaling or release cadence." That signals judgment over hype.

⚠️ Trade-off / Gotcha

Migration goes *both* ways in 2026 — several high-profile teams have consolidated over-split microservices back into modular monoliths to cut latency and ops cost. "More services" is not "more mature." The right answer is always "it depends on the constraints," and naming the constraints is the skill.

🔥 Want the full TechTribers library?

Java 8→21, Spring Boot, System Design, Kafka, DSA & more — all in one bundle.

www.techtriber.com

SECTION 02

Communication

How services talk: synchronous vs asynchronous, REST vs gRPC vs messaging, API versioning in Spring Framework 7, idempotency, and correlation IDs.



Q9 Synchronous vs asynchronous communication — when do you use each?

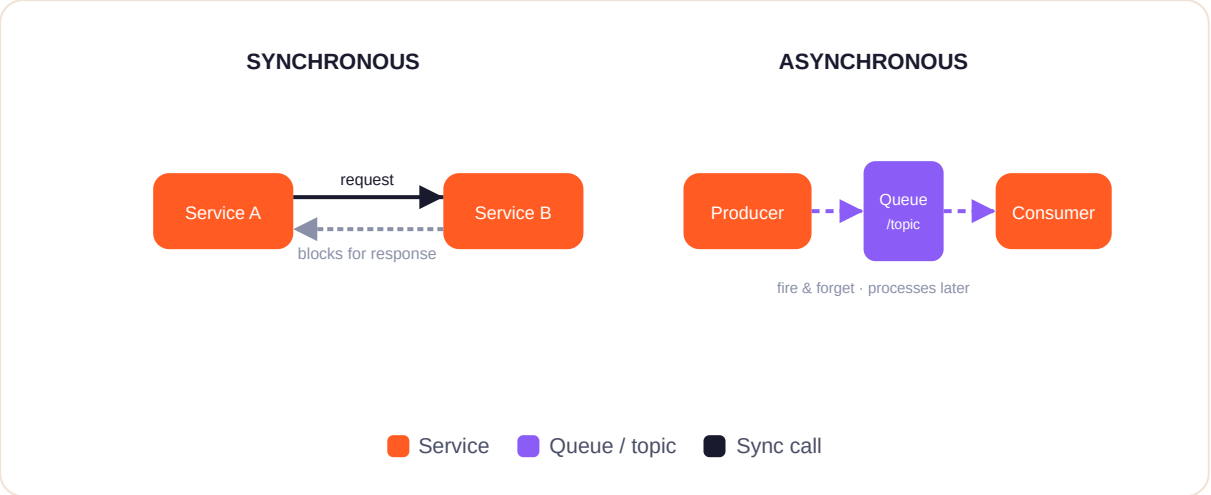
🟢 Beginner

Communication

💡 Explanation

Synchronous (REST, gRPC): the caller sends a request and blocks waiting for a response. Simple and immediate, but it couples availability — if the callee is down or slow, the caller is too.
Asynchronous (messaging/events via Kafka, RabbitMQ): the sender publishes a message and moves on; consumers process it when ready. This decouples services in time and improves resilience, at the cost of eventual consistency and harder reasoning.
Rule of thumb: use **sync** when the caller genuinely needs the answer now to continue (e.g. "is this user authorized?"). Use **async** for anything that can happen in the background (e.g. "send a confirmation email," "update the search index").

📐 Diagram



⚠️ Trade-off / Gotcha

Don't reach for sync calls just because they're easier to reason about. A chain of synchronous calls couples the availability of every link — one slow service creates a cascading slowdown. When in doubt for non-critical-path work, prefer events.

Q10 REST vs gRPC vs messaging — how do you choose between them?

🟡 Intermediate

Communication

💡 Explanation

- **REST/JSON over HTTP** — universal, human-readable, easy to debug and cache. The default for public APIs and most service-to-service calls. Higher payload size and parsing cost than binary.
- **gRPC** — binary Protobuf over HTTP/2, with code-gen contracts and streaming. Much faster and more compact; ideal for high-throughput internal service-to-service calls. Less browser-friendly and harder to debug by hand.
- **Messaging (Kafka/RabbitMQ)** — asynchronous, decoupled, durable. Best for events, fan-out, and work that shouldn't block the caller.

🎯 Scenario

"Internal services exchange millions of small messages per minute and JSON parsing is a bottleneck." That's a strong gRPC case — the binary contract and HTTP/2 multiplexing cut both payload size and latency. But if the same data also needs to reach browsers or third parties, you may expose REST at the edge (via the gateway) and use gRPC internally.

⚠️ Trade-off / Gotcha

gRPC's tight Protobuf contracts are a double-edged sword: fast, but you must manage schema evolution carefully (backward/forward compatibility), or a contract change breaks consumers. REST's looseness tolerates additive change more gracefully.

Q11 How do you version a service API? (incl. Spring Framework 7 API versioning)

🟡 Intermediate

Spring Boot 4

💡 Explanation

API versioning lets you evolve a service without breaking existing consumers. Common strategies: **URI** (`/v2/orders`), **header** (`X-API-Version: 2`), **media type** (`Accept: application/vnd.api.v2+json`), or a **query param**. Spring Framework 7 (Spring Boot 4) introduced **first-class API versioning**, so you declare the version directly on the mapping and Spring routes to the right handler — and Spring Cloud Gateway 5.0 can route by version at the edge.

⌨️ Code · Spring Framework 7 API versioning

OrderController.java

```java
@RestController
@RequestMapping("/orders")
public class OrderController {

  // Served to clients requesting version 1
  @GetMapping(version = "1")
  public OrderV1 getV1(@PathVariable String id) { ... }

  // Same path, version 2 — Spring routes by the requested version
  @GetMapping(version = "2")
  public OrderV2 getV2(@PathVariable String id) { ... }
}

// Configure how the version is resolved (header, query, or media type)
@Override
public void configureApiVersioning(ApiVersionConfigurer cfg) {
  cfg.useRequestHeader("X-API-Version");
}
```

⚠️ Trade-off / Gotcha

Versioning is easy to add and painful to retire. Every version you publish is a version you must support and test. Prefer **backward-compatible additive changes** (new optional fields) over new versions whenever possible — and have a deprecation policy so old versions actually get removed.

Q12 What is idempotency and why is it critical in microservices?

🔴 Advanced | Reliability

💡 Explanation

An operation is **idempotent** if performing it multiple times has the same effect as performing it once. This matters because networks force **retries** — a client times out, retries, but the original request actually succeeded. Without idempotency, a retried "charge card" becomes a double charge.

The standard technique is an **idempotency key**: the client sends a unique key with the request; the server records it and, on a duplicate key, returns the original result instead of reprocessing.

⌨️ Code · idempotent endpoint

PaymentController.java

```java
@PostMapping("/payments")
public ResponseEntity<Payment> pay(
    @RequestHeader("Idempotency-Key") String key,
    @RequestBody PaymentRequest req) {

  // Already processed this key? Return the original result.
  return store.findByKey(key)
      .map(p -> ResponseEntity.ok(p))
      .orElseGet(() -> {
        Payment p = paymentService.charge(req);
        store.save(key, p);   // persist key + result atomically
        return ResponseEntity.ok(p);
      });
}
```

⚠️ Trade-off / Gotcha

GET, PUT, and DELETE are naturally idempotent; **POST is not**. Any "create" or "charge" endpoint that can be retried needs explicit idempotency handling. And the key + result must be stored atomically with the side effect — otherwise a crash between "charge" and "save key" reintroduces the double-processing you were trying to prevent.

Q13 How do correlation IDs work and why do you need them?

🟡 Intermediate

Observability

💡 Explanation

A **correlation ID** (or trace ID) is a unique identifier generated at the edge (gateway) and propagated through every service involved in handling a single request — passed along in headers and included in every log line. Without it, a request that touches six services produces six disconnected log streams and debugging is guesswork. With it, you can filter all logs by one ID and see the entire journey.

In Spring Boot, **Micrometer Tracing** handles this automatically: it generates trace/span IDs, propagates them across RestClient/WebClient/messaging calls, and adds them to the MDC so they appear in logs — feeding tools like Zipkin, Tempo, or Jaeger.

⌨️ Code · Micrometer Tracing

application.yml + log pattern

```yaml
# Trace IDs flow automatically across services via Micrometer Tracing
management:
  tracing:
    sampling:
      probability: 1.0     # sample 100% in dev, lower in prod

logging:
  pattern:
    # [traceId, spanId] injected into every log line
    level: "%5p [${spring.application.name},%X{traceId:-},%X{spanId:-}]"
```

🎯 Scenario

"A customer reports their order failed at 14:32." With correlation IDs you grab the trace ID from the gateway access log, filter every service by it, and watch the request flow until you hit the service that threw — minutes instead of hours. The interview follow-up is usually "how does the ID get from service to service?" — answer: header propagation, automated by Micrometer Tracing's instrumentation.

⚠️ Trade-off / Gotcha

Propagation breaks at boundaries you forget to instrument — a raw thread pool, a manually-built HTTP client, or a message queue without the tracing headers. The chain is only as complete as its least-instrumented hop.

Q14 How do services authenticate each other (service-to-service auth)?

🔴 Advanced | Security

💡 Explanation

Internal calls still need identity and authorization — "it's behind the firewall" is not security (zero-trust). Common approaches:

- **Propagated user token (JWT)** — the caller forwards the end-user's access token; the callee validates it as an OAuth2 resource server. Good when the downstream needs the user's identity/scopes.
- **Client credentials (OAuth2)** — the service authenticates *as itself* with its own token, for machine-to-machine actions with no user context.
- **mTLS** — mutual TLS where both sides present certificates; often handled by a service mesh, proving service identity at the transport layer.

In Spring Security 7, the callee is configured as a resource server that validates the JWT (signature, issuer, audience, expiry) and maps scopes to authorities.

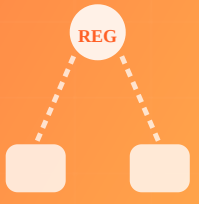
⚠️ Trade-off / Gotcha

Blindly forwarding the user's token to every downstream is convenient but over-shares scope and creates a token-replay surface. For pure machine actions, prefer client-credentials tokens scoped to exactly what that service may do. And never trust a header like `X-User-Id` set by an upstream service unless the network path guarantees it can't be spoofed.

SECTION 03

Service Discovery & Routing

How services find each other in a world where instances come and go — discovery, the lb:// scheme, Spring Cloud Gateway 5.0, and the BFF pattern.



Q15 What is service discovery, and what's the difference between client-side and server-side discovery?

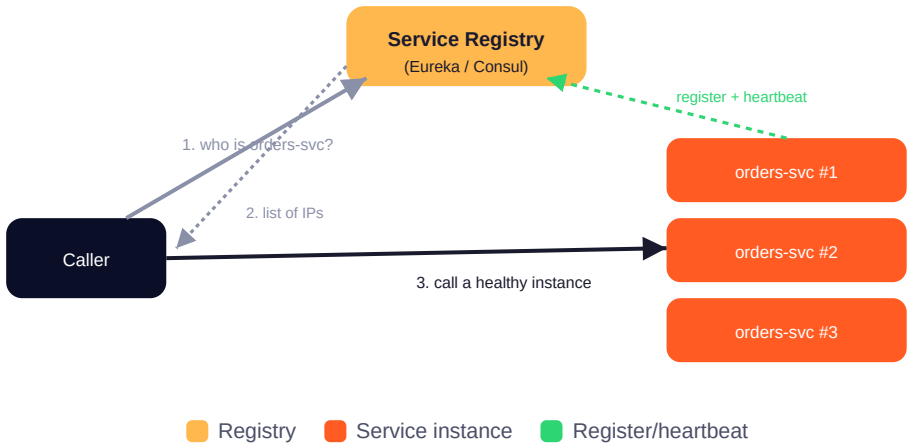
🟢 Beginner | Discovery

💡 Explanation

In a dynamic environment, service instances start, stop, scale, and move constantly — their IPs aren't fixed. **Service discovery** lets a caller find a healthy instance of a service by *name* instead of a hard-coded address. Instances register themselves with a **service registry** (e.g. Eureka, Consul, or Kubernetes' built-in DNS); callers query it to resolve a name to a live address.

- **Client-side discovery** — the caller queries the registry and picks an instance itself (often with a client-side load balancer like Spring Cloud LoadBalancer). More control, but every client needs discovery logic.
- **Server-side discovery** — the caller hits a fixed endpoint (a load balancer or gateway) that does the lookup and routing. Simpler clients; the routing tier becomes critical infrastructure.

📐 Diagram



⚠️ Trade-off / Gotcha

In Kubernetes you often **don't need** Eureka at all — K8s Services + DNS provide server-side discovery out of the box. Bolting Eureka on top of K8s is a common redundancy. Know your platform before adding a registry.

Q16 How does Eureka work, and how do you register a Spring Boot service with it?

🟡 Intermediate

Spring Cloud

💡 Explanation

Eureka (Spring Cloud Netflix) is a registry where services register on startup and send periodic **heartbeats**. If heartbeats stop, Eureka evicts the instance so callers stop routing to it. Clients fetch and cache the registry, enabling client-side discovery. Eureka favors **availability over consistency** (AP in CAP terms) — under a partition it keeps serving possibly-stale data rather than going down, which is usually the right call for discovery.

⌨️ Code · Eureka client (Spring Cloud 2025.1)

pom.xml + application.yml

```
<!-- Spring Cloud 2025.1 (Oakwood), modules at 5.0.x -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

# application.yml
spring:
  application:
    name: orders-service       # this is the discoverable name
eureka:
  client:
    service-url:
      defaultZone: http://eureka:8761/eureka
  instance:
    prefer-ip-address: true
```

▶️ Result

On startup the service appears in the Eureka dashboard as `ORDERS-SERVICE` with its instances. Other services can now resolve it by that name rather than an IP — and as you scale to 3 instances, all three register automatically.

⚠️ Trade-off / Gotcha

Eureka's client-side cache means there's a propagation lag — a just-killed instance may linger in caches for a few seconds, so callers can still hit a dead address. This is exactly why you pair discovery with **retries and circuit breakers** (Section 04): discovery gets you *close*, resilience handles the stale-instance window.

Q17 What is the lb:// scheme and how does client-side load balancing work in Spring Cloud?

🟡 Intermediate

Spring Cloud

💡 Explanation

Spring Cloud LoadBalancer is the client-side load balancer (it replaced the old Netflix Ribbon). When you call a URL with the `lb://` scheme — e.g. `lb://orders-service` — it resolves the service name via discovery, gets the list of healthy instances, and picks one (round-robin by default). In Spring Cloud Commons 5.0 you can put `lb://` base URLs directly on annotated **Spring Interface Clients**.

⌨️ Code · load-balanced RestClient + Interface Client

ClientConfig.java

```
// Classic: a load-balanced RestClient
@Bean
@LoadBalanced
RestClient.Builder restClientBuilder() {
  return RestClient.builder();
}
// call by service name, not IP:
// restClient.get().uri("lb://orders-service/orders/{id}", id)...

// Spring Cloud Commons 5.0: lb:// directly on an interface client
@HttpExchange(url = "lb://orders-service")
interface OrdersClient {
  @GetExchange("/orders/{id}")
  Order getOrder(@PathVariable String id);
}
```

⚠️ Trade-off / Gotcha

Client-side load balancing means each caller makes its own routing decision from its own (possibly stale) instance list — there's no central view. Round-robin also ignores instance load; if one instance is slow, naive round-robin keeps sending it traffic. For load-aware routing you need a smarter algorithm or a server-side LB/mesh.

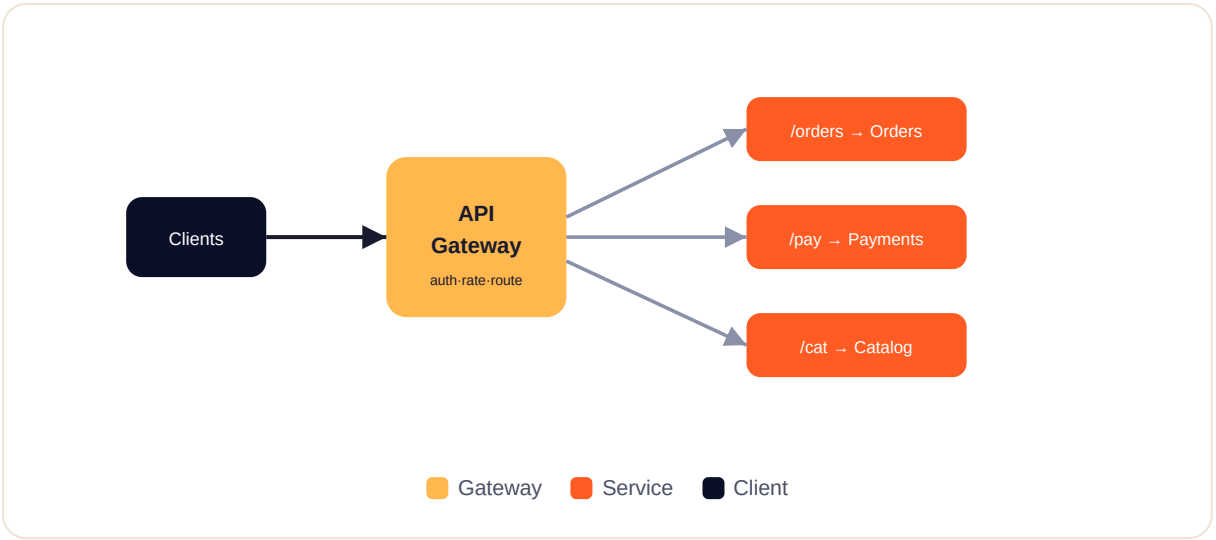
Q18 What is an API Gateway and why put one in front of your services?

🟢 Beginner | Gateway

💡 Explanation

An **API Gateway** is the single entry point for clients into the system. Instead of clients knowing about dozens of services, they hit the gateway, which routes to the right service and handles cross-cutting concerns in one place: authentication, rate limiting, TLS termination, request/response transformation, API versioning, and CORS. **Spring Cloud Gateway 5.0** (part of the 2025.1 train) is the Spring option, available in reactive (WebFlux) and servlet (WebMVC) flavors.

📐 Diagram



⚠️ Trade-off / Gotcha

The gateway is a single point of failure and a potential bottleneck — it must be highly available and horizontally scaled. Also resist stuffing **business logic** into it; the gateway is for cross-cutting concerns. Logic in the gateway becomes a hidden distributed monolith that every team trips over.

Q19 How do routes, predicates, and filters work in Spring Cloud Gateway 5.0?

🟡 Intermediate | Gateway

💡 Explanation

A Gateway **route** has three parts: **predicates** (conditions that decide if a request matches — path, header, method, host, or the new API **version** predicate), **filters** (transform the request/response — add headers, strip prefixes, rate-limit, circuit-break), and a **URI** target (often `lb://service-name` for load-balanced routing). Gateway 5.0 ships distinct `server-webflux` and `server-webmvc` artifacts — pick reactive or servlet to match your stack.

⌨️ Code · Gateway routes (Spring Cloud Gateway 5.0)

application.yml

```yaml
spring:
  cloud:
    gateway:
      server:
        webflux:
          routes:
            - id: orders-route
              uri: lb://orders-service      # load-balanced target
              predicates:
                - Path=/api/orders/**
                - Method=GET,POST
              filters:
                - StripPrefix=1             # /api/orders → /orders
                - CircuitBreaker=ordersCB   # resilience at the edge
```

🎯 Scenario

"Route v2 of the orders API to a new service while v1 still hits the old one." With Gateway 5.0's API-version predicate you match on the requested version and send each to the right upstream — a clean canary/migration lever without touching clients.

⚠️ Trade-off / Gotcha

The note in the Spring Cloud 2025.x release: `X-Forwarded-*` header handling is disabled by default unless you configure `trusted-proxies`. If your downstream relies on forwarded client IPs, set the trusted-proxies regex or you'll lose them.

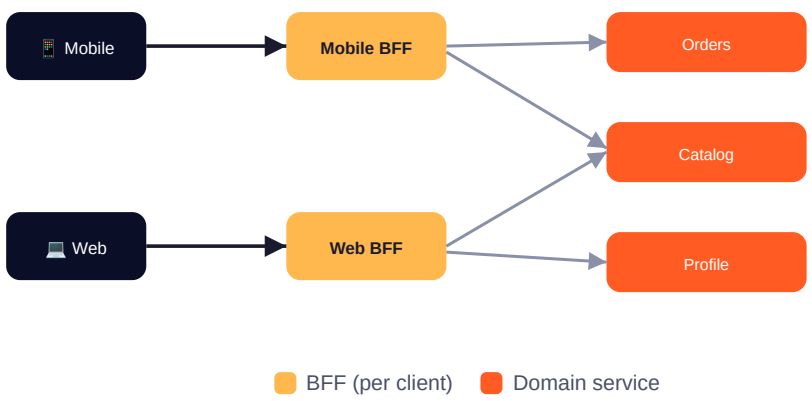
Q20 What is the Backend-for-Frontend (BFF) pattern?

🔴 Advanced | Gateway

💡 Explanation

A **BFF** is a dedicated gateway/aggregation layer per client type — one for web, one for mobile, maybe one for partners. Each BFF tailors and aggregates responses to exactly what that client needs, instead of forcing one generic API to serve wildly different consumers. A mobile BFF might bundle three service calls into one slim payload to save battery and bandwidth; a web BFF might return a richer shape.

📐 Diagram



⚠️ Trade-off / Gotcha

BFFs add real value but also duplication — shared logic can drift across BFFs, and you now have more deployables to own. Use them when client needs genuinely diverge; a single gateway is fine when they don't. And keep the BFF an *aggregation* layer, not a place where core business rules quietly migrate.

SECTION 04

Resilience

The network will fail — design for it. Circuit breakers, retries, timeouts, bulkheads, rate limiting, and the new declarative resilience in Spring Cloud Commons 5.0.



Q21 What is a circuit breaker, and what are its three states?

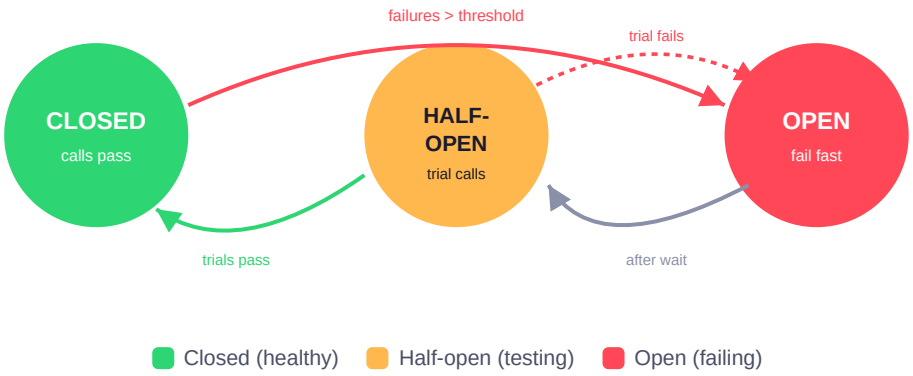
🟡 Intermediate | Resilience

💡 Explanation

A **circuit breaker** stops a service from hammering a failing dependency, giving it room to recover and failing fast instead of piling up blocked threads. Like an electrical breaker, it has three states:

- **CLOSED** — normal; calls pass through. Failures are counted.
- **OPEN** — failure threshold exceeded; calls are rejected immediately (fail fast / fallback) without touching the dependency. After a wait, it moves to half-open.
- **HALF-OPEN** — a few trial calls are allowed. If they succeed, close the circuit; if they fail, open it again.

📐 Diagram · circuit breaker state machine



⚠️ Trade-off / Gotcha

A circuit breaker doesn't *fix* the failure — it contains it. You still need a **fallback** (cached value, default, queued retry, or a graceful error) for when it's open. And thresholds need tuning: too sensitive and it trips on normal blips; too lax and it never protects you.

Q22 How do retries and timeouts work together — and how do you avoid a retry storm?

🔴 Advanced | Resilience

💡 Explanation

Every remote call needs a **timeout** — without one, a hung dependency ties up your threads until you fall over. **Retries** handle transient failures (a brief blip, a dropped packet), but naive retries are dangerous: if a service is struggling, everyone retrying at once multiplies the load and finishes it off — a **retry storm**. The fixes: **exponential backoff** (wait longer each attempt), **jitter** (randomize the wait so clients don't retry in lockstep), a small **retry cap**, and only retrying **idempotent** operations.

⌨️ Code · Resilience4j retry with backoff + jitter

application.yml

```yaml
resilience4j:
  retry:
    instances:
      ordersRetry:
        max-attempts: 3
        wait-duration: 200ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2
        enable-randomized-wait: true      # jitter
  timelimiter:
    instances:
      ordersRetry:
        timeout-duration: 2s              # hard timeout per call
```

⚠️ Trade-off / Gotcha

Retries + non-idempotent operations = duplicate side effects (double charges, double orders). Only retry reads or operations protected by an idempotency key (Q12). Also watch **nested retries**: if A retries B which retries C, you get a multiplicative explosion — decide at *one* layer who owns the retry.

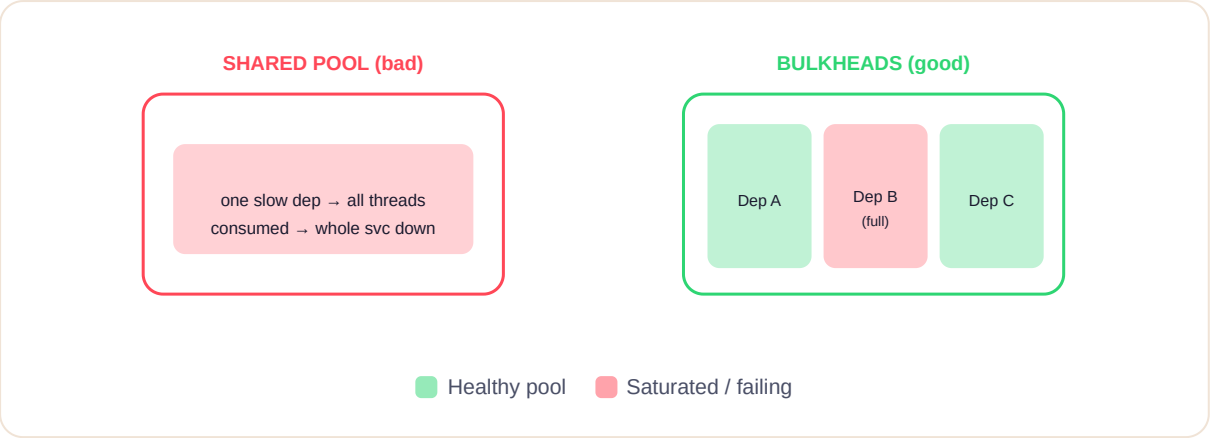
Q23 What is the bulkhead pattern?

🟡 Intermediate | Resilience

💡 Explanation

Named after the watertight compartments in a ship's hull, the **bulkhead** pattern isolates resources so a failure in one area can't sink the whole service. Concretely: give each downstream dependency its own bounded pool of threads (or concurrent-call permits). If dependency A becomes slow and exhausts *its* pool, calls to B and C still have their own capacity and keep working — instead of one slow dependency consuming every thread in a shared pool and taking the entire service down.

📐 Diagram



⚠️ Trade-off / Gotcha

Bulkheads cost capacity — partitioning threads into per-dependency pools means lower peak utilization than one big shared pool. That's the price of isolation. Size the pools from real traffic, or a too-small bulkhead becomes its own bottleneck under normal load.

Q24 How does rate limiting work — token bucket vs leaky bucket?

🟡 Intermediate | Resilience

💡 Explanation

Rate limiting caps how many requests a client can make in a window, protecting services from overload and abuse. Two classic algorithms:

- **Token bucket** — tokens refill at a steady rate into a bucket of fixed capacity; each request spends a token. Allows short **bursts** (spend accumulated tokens) while bounding the long-run average. Most common for APIs.
- **Leaky bucket** — requests enter a queue that drains ("leaks") at a fixed rate; overflow is rejected. Produces a **smooth, constant** output rate, no bursts.

Spring Cloud Gateway ships a Redis-backed token-bucket rate limiter you apply as a route filter — Redis makes the limit consistent across all gateway instances.

⚠️ Trade-off / Gotcha

In a multi-instance gateway, an **in-memory** limiter lets each instance allow the full quota — three instances = 3× the intended limit. Rate limits must use shared state (Redis) to be enforced globally. Also decide your limit *key* carefully: per-user, per-IP, and per-API-key each have very different abuse profiles.

Q25 What is declarative resilience on Spring Interface Clients (Spring Cloud Commons 5.0)?

🔴 Advanced | Spring Cloud

💡 Explanation

A 2025.1 (Oakwood) addition: **Spring Cloud Commons 5.0** adds built-in **circuit-breaker integration for annotated Spring Interface Clients** (the declarative HTTP clients backed by RestClient/WebClient). Instead of manually wrapping every call in Resilience4j code, you declare the client interface and resilience is wired in via configuration — less boilerplate, consistent behavior across clients. It pairs with the `lb://` scheme so one interface gets both load balancing and circuit breaking.

⌨️ Code · resilient interface client

OrdersClient.java

```
// Declarative HTTP client — load-balanced + circuit-broken
@HttpExchange(url = "lb://orders-service")
interface OrdersClient {

  @GetExchange("/orders/{id}")
  Order getOrder(@PathVariable String id);
}

# application.yml — attach a circuit breaker by name
resilience4j:
  circuitbreaker:
    instances:
      ordersClient:
        failure-rate-threshold: 50       # open at 50% failures
        sliding-window-size: 20
        wait-duration-in-open-state: 10s
```

⚠️ Trade-off / Gotcha

Declarative resilience is cleaner but **hides behavior** — a new engineer may not realize a call can be short-circuited and return a fallback. Document which clients are protected and what their fallbacks return, or you'll get "why did this return stale/empty data?" debugging sessions.

Q26 What is graceful degradation, and how does it differ from failing hard?

🟡 Intermediate | Resilience

💡 Explanation

Graceful degradation means the system keeps delivering its *core* value when a non-critical dependency fails, instead of returning an error for the whole request. If the recommendations service is down, an e-commerce product page should still render the product, price, and "buy" button — just without the "you might also like" strip. The user barely notices; the alternative (a 500 on the whole page because one widget failed) loses the sale.

It's implemented with the patterns in this section working together: a circuit breaker detects the failing dependency, a **fallback** supplies a sensible default (empty list, cached data, a hidden section), and the request completes.

🎯 Scenario

"During a traffic spike, the personalization service gets overwhelmed." With graceful degradation, the circuit opens, the homepage falls back to a generic (cached) layout for everyone, and the site stays up. Once personalization recovers, the circuit closes and tailored content returns — no outage, no incident page.

⚠️ Trade-off / Gotcha

Degradation has to be **designed and tested** — you must decide up front which features are core vs optional, and actually exercise the fallback paths (chaos testing). A fallback that's never tested is just an untested code path that fires for the first time during your worst incident.

SECTION 05

Data & Transactions

The hardest part of microservices. Database-per-service, distributed transactions, the Saga pattern, the Outbox pattern, CQRS, event sourcing, and eventual consistency.



Q27 Why database-per-service? And what's wrong with a shared database?

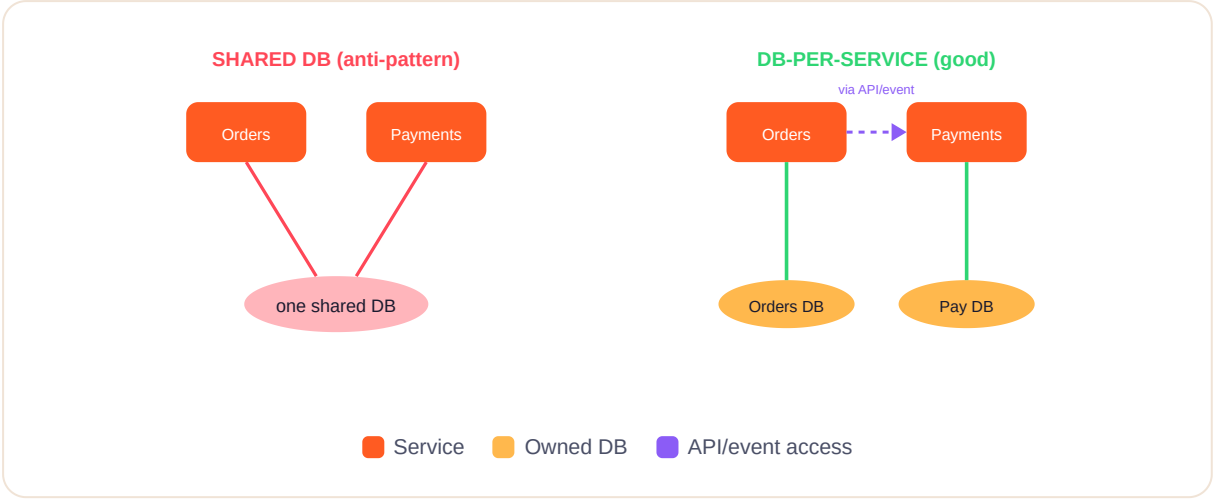
🟡 Intermediate | Data

💡 Explanation

The **database-per-service** rule says each service owns its data and is the *only* thing that touches its database; others get to it through the service's API, never the tables directly. This preserves the independence microservices exist for — a service can change its schema, switch databases, or scale storage without coordinating with anyone.

A **shared database** quietly destroys that. The moment two services read/write the same tables, they're coupled at the schema level: one team's migration breaks another team's queries, you can't deploy independently, and you've built a distributed monolith with extra network hops.

📐 Diagram



⚠️ Trade-off / Gotcha

DB-per-service is the right default, but it makes **cross-service queries and transactions hard** — you can no longer do a JOIN or an ACID transaction across two services' data. That difficulty is exactly what the next questions (saga, CQRS, eventual consistency) exist to solve. The shared DB "solves" it only by reintroducing coupling.

Q28 Why can't you use a normal ACID transaction across microservices?

🟡 Intermediate

Transactions

💡 Explanation

A local ACID transaction works because one database controls one commit. Across services with separate databases, there's no shared transaction manager to atomically commit all of them. The old answer — **two-phase commit (2PC)** with a distributed coordinator — is generally avoided in microservices: it's slow, it holds locks across the network, and it blocks everything if the coordinator fails (low availability, poor scalability).

So microservices trade **strong consistency for availability** and use **eventual consistency** via patterns like the Saga (Q29): a sequence of local transactions, each in its own service, coordinated by events or an orchestrator, with compensating actions to undo on failure.

⚠️ Trade-off / Gotcha

The interview trap is reaching for 2PC to "keep it consistent." Senior signal: explain *why* 2PC doesn't fit microservices (blocking, coordinator as SPOF, locks held across network calls) and that the real answer is embracing eventual consistency with sagas — accepting a brief inconsistency window in exchange for availability.

Q29 What is the Saga pattern? Explain orchestration vs choreography.

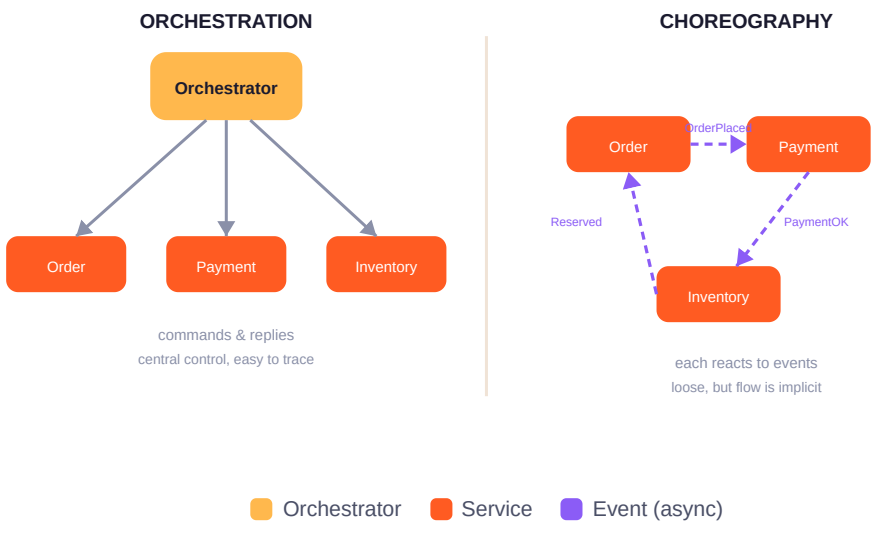
🔴 Advanced | Saga

💡 Explanation

A **Saga** manages a business transaction that spans multiple services as a sequence of **local transactions**. Each step commits in its own service and publishes an event/triggers the next. If a step fails, the saga runs **compensating transactions** that semantically undo the previous steps (e.g. refund the payment, release the reserved stock). It gives you a consistent outcome without a distributed lock.

- **Orchestration** — a central *orchestrator* tells each service what to do and tracks progress. Explicit, easy to follow and monitor; the orchestrator is a coordination point you must build and own.
- **Choreography** — no coordinator; each service reacts to events and emits its own. Loosely coupled and resilient, but the end-to-end flow is implicit and harder to trace.

📐 Diagram · Orchestration vs Choreography



🎯 Scenario

"Place order → charge payment → reserve stock. Payment succeeds but stock is sold out." The saga fires a **compensating transaction**: refund the payment and cancel the order, leaving the system consistent. The interviewer's follow-up: "What if the refund itself fails?" — answer: compensations must be retryable and idempotent, and you need a dead-letter / manual-intervention path for the rare stuck case.

⚠️ Trade-off / Gotcha

Choreography looks elegant for 2–3 services but becomes a tangle of event chains nobody can trace as it grows ("event spaghetti"). Orchestration scales better for complex flows

because the logic lives in one visible place. Pick choreography for simple flows, orchestration when there are many steps or you need clear monitoring.

Q30 What is the dual-write problem, and how does the Outbox pattern solve it?

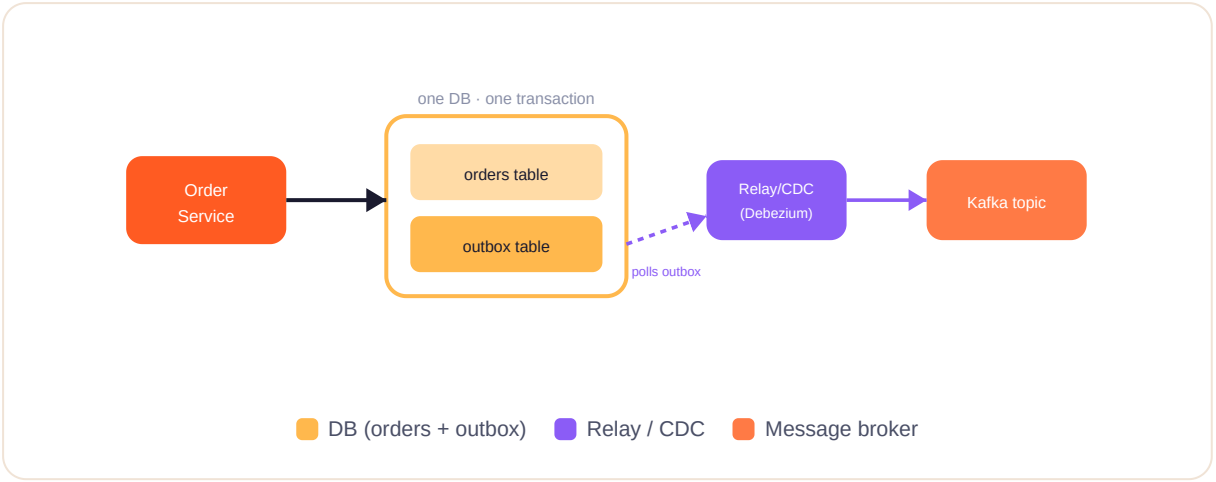
🔴 Advanced | Outbox

💡 Explanation

The **dual-write problem**: a service often needs to do two things in one step — update its database *and* publish an event ("order saved" → publish `OrderPlaced`). These are two separate systems (DB + broker) with no shared transaction. If the DB commit succeeds but the publish fails (or vice versa), you get inconsistency: an order exists with no event, or an event with no order.

The **Outbox pattern** fixes this by writing the event into an **outbox table in the same database**, inside the *same local transaction* as the business change. A separate relay process (or Change Data Capture, e.g. Debezium) then reads the outbox and publishes to the broker. Now the business write and the "intent to publish" commit atomically — the event can't be lost.

📐 Diagram · Outbox + relay



⚠️ Trade-off / Gotcha

Outbox guarantees **at-least-once** delivery, not exactly-once — the relay may publish a message twice (e.g. it crashes after publishing but before marking the row done). So **consumers must be idempotent** (Q12). Outbox solves the producer side of reliability; idempotent consumers close the loop.

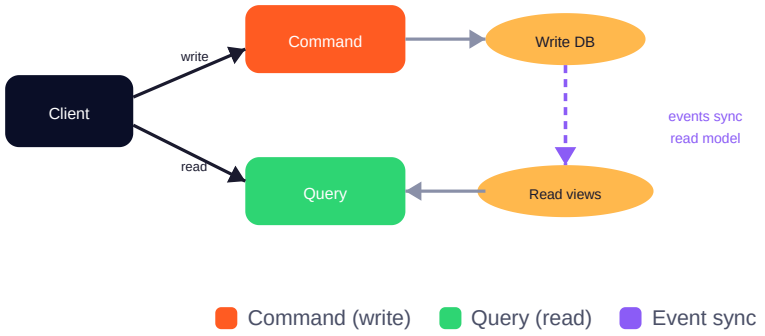
Q31 What is CQRS and when should you use it?

🔴 Advanced | CQRS

💡 Explanation

CQRS (Command Query Responsibility Segregation) separates the **write model** (commands that change state) from the **read model** (queries that return data). They can use different schemas, even different databases — a normalized store for writes, denormalized/indexed views optimized for reads. The read side is kept in sync, usually asynchronously, via events from the write side.
It shines when reads and writes have very different shapes or scale needs: e.g. a system with heavy, complex read queries (dashboards, search) and simpler writes, where one schema can't serve both well.

📐 Diagram



⚠️ Trade-off / Gotcha

CQRS adds significant complexity (two models, sync logic, eventual consistency between them). It's frequently **over-applied**. Don't reach for it by default — use it only where the read/write asymmetry genuinely justifies it. For most services, one model is correct.

Q32 What is event sourcing?

🔴 Advanced

Event Sourcing

💡 Explanation

Instead of storing the *current state* of an entity, **event sourcing** stores the full sequence of **events** that led to it (e.g. `AccountOpened`, `MoneyDeposited`, `MoneyWithdrawn`). Current state is derived by replaying those events. You get a complete, immutable audit log for free, the ability to reconstruct state at any past point in time, and a natural fit with CQRS (events build the read views).

⚠️ Trade-off / Gotcha

It's powerful but heavy: querying current state means replaying events (so you need **snapshots** for performance), **schema evolution** of old events is genuinely hard, and the mental model is unfamiliar to most teams. Reserve it for domains where the audit trail and temporal queries are first-class requirements (finance, ledgers) — not as a default storage strategy.

Q33 What is eventual consistency, and how do you design a good user experience around it?

🟡 Intermediate

Consistency

💡 Explanation

Eventual consistency means that after a write, different parts of the system become consistent *after some delay*, not instantly — but given no new updates, they'll converge. It's the natural state of a system using async events and DB-per-service. The engineering challenge is mostly UX: users expect to see their own change immediately.
Techniques: **read-your-own-writes** (route a user's reads to the source they just wrote, or return the new value optimistically), **optimistic UI** (show the change instantly, reconcile in the background), and clear **pending states** ("processing…") so the delay is communicated rather than hidden.

🎯 Scenario

"User posts a comment; it's written to the write store but the read view updates 200ms later, so a refresh shows it missing." Fix with optimistic UI — render the comment immediately on the client while the event propagates — so the user never sees the gap, even though the system is eventually consistent underneath.

⚠️ Trade-off / Gotcha

Don't claim eventual consistency where you actually need strong consistency. Money movements, inventory at zero stock, and uniqueness constraints (one username) often demand a strongly-consistent check at the decision point. Match the consistency model to the business risk of being briefly wrong.

🔥 The complete TechTribers bundle

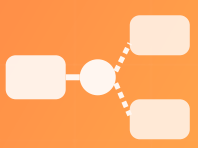
This guide is one of many — Java 8→21, Spring Boot, System Design, Kafka, DSA & more.

www.techtriber.com

SECTION 06

Event-Driven & Messaging

The async backbone of resilient microservices. Event-driven architecture, Kafka vs RabbitMQ, Spring Cloud Stream 5.0, pub-sub, and delivery guarantees.



Q34 What is event-driven architecture, and what are its benefits and costs?

🟢 Beginner | EDA

💡 Explanation

In **event-driven architecture**, services communicate by producing and consuming **events** ("something happened") rather than calling each other directly. A producer emits an event to a broker and doesn't know or care who consumes it; consumers subscribe and react. This **decouples** services in time (the consumer can be down and catch up later) and in knowledge (the producer doesn't depend on consumers).

Benefits: loose coupling, independent scaling, resilience (broker buffers during spikes/outages), and easy fan-out (many consumers of one event). Costs: eventual consistency, harder debugging (no single call stack), and the broker becomes critical infrastructure.

⚠️ Trade-off / Gotcha

"Event-driven" is not free decoupling — you trade an explicit, traceable request flow for an implicit one. Without good observability (correlation IDs, tracing, a schema registry), an event-driven system becomes a black box where nobody can answer "what happens when an order is placed?" Invest in that tooling before going heavily event-driven.

Q35 Kafka vs RabbitMQ — when do you choose each?

🟡 Intermediate | Messaging

💡 Explanation

Both move messages between services, but they're built for different jobs:

- **Kafka** — a distributed, partitioned **log**. Messages are retained and replayable; consumers track their own offset. Built for high-throughput event streaming, event sourcing, and multiple independent consumer groups reading the same stream. Think "durable river of events."
- **RabbitMQ** — a traditional **message broker** with rich routing (exchanges, queues, bindings). Messages are typically consumed and removed. Great for task queues, complex routing, request/reply, and per-message acknowledgement. Think "smart post office."

▶️ Quick guide

Choose Kafka for event streaming, replay, analytics pipelines, high throughput, and many consumers of the same data. **Choose RabbitMQ** for work queues, sophisticated routing, lower-volume command messaging, and when you want the broker to manage delivery/ack per message.

⚠️ Trade-off / Gotcha

Don't use Kafka as a simple task queue — its consumer-group/partition model and offset management are overkill and awkward for "one worker grabs one job." Conversely, don't use RabbitMQ when you need to replay history or have many independent readers of the same stream; messages are gone once acked.

Q36 How does Spring Cloud Stream 5.0 abstract messaging? Show a producer and consumer.

🟡 Intermediate

Spring Cloud

💡 Explanation

Spring Cloud Stream (5.0, part of the 2025.1 train) lets you write messaging code against a **binder** abstraction instead of broker-specific APIs. You define producers/consumers as plain `Supplier`, `Function`, or `Consumer` beans; a binder (Kafka, RabbitMQ, etc.) connects them to real destinations via configuration. Swap brokers by changing the binder dependency and config — your business code stays the same.

⌨️ Code · functional producer + consumer

MessagingConfig.java + application.yml

```
// Consumer: a Consumer<T> bean is auto-bound to an input destination
@Bean
Consumer<OrderPlaced> handleOrder() {
  return event -> {
    log.info("Order placed: {}", event.orderId());
    inventory.reserve(event);
  };
}

// Producer: send via StreamBridge from anywhere
streamBridge.send("orders-out-0", new OrderPlaced(id));

# application.yml — bind functions to Kafka topics
spring:
  cloud:
    function:
      definition: handleOrder
    stream:
      bindings:
        handleOrder-in-0:  { destination: orders.placed, group: inventory }
        orders-out-0:      { destination: orders.placed }
```

⚠️ Trade-off / Gotcha

The binder abstraction is convenient but **leaky** — Kafka and RabbitMQ have genuinely different semantics (partitions/offsets vs queues/acks). For advanced features you'll still drop to broker-specific config, and "just swap the binder" rarely works seamlessly for non-trivial apps. Treat it as reducing boilerplate, not as true broker portability.

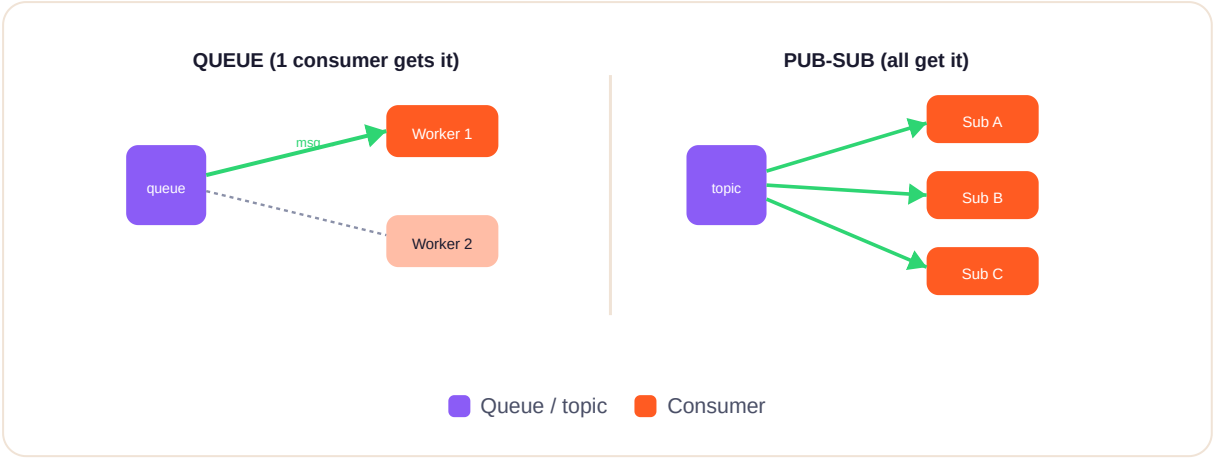
Q37 What's the difference between a queue (point-to-point) and pub-sub?

🟢 Beginner | Messaging

💡 Explanation

Point-to-point (queue): each message is delivered to exactly *one* consumer. Multiple workers can read from the same queue, but any given message goes to only one of them — ideal for distributing work (a task queue). **Publish-subscribe (topic):** each message is delivered to *all* subscribers. One event, many independent reactions — ideal for broadcasting that "something happened."
In Kafka, a **consumer group** elegantly gives you both: within one group, a partition's messages go to one consumer (queue-like load balancing); across different groups, every group gets all messages (pub-sub).

📐 Diagram



⚠️ Trade-off / Gotcha

A frequent Kafka mistake: putting every consumer in the *same* group expecting all to receive every message — instead they split the partitions and each message hits only one. If you want independent fan-out (pub-sub), give each logical consumer its **own group id**.

Q38 At-least-once vs at-most-once vs exactly-once delivery — what's realistic?

🔴 Advanced | Delivery

💡 Explanation

- **At-most-once** — fire and forget; a message may be lost but never duplicated. Lowest overhead, acceptable only when occasional loss is fine (e.g. some metrics).
- **At-least-once** — retried until acknowledged; never lost but may be **duplicated**. The practical default for most systems — paired with idempotent consumers.
- **Exactly-once** — each message effectively processed once, no loss, no duplicates. The hardest to achieve and often misunderstood.

"Exactly-once" in distributed systems is really **effectively-once**: you combine at-least-once delivery with idempotent processing (or transactional/dedup mechanisms) so duplicates have no visible effect. Kafka offers transactional, exactly-once *semantics* within Kafka-to-Kafka pipelines, but the moment you touch an external system, you fall back to "at-least-once + idempotency."

⚠️ Trade-off / Gotcha

Chasing true exactly-once everywhere is a classic time sink. The senior answer: **design for at-least-once and make consumers idempotent.** That gives you the practical guarantees of exactly-once with far less complexity. Reserve Kafka's transactional EOS for closed Kafka pipelines where it genuinely applies.

Q39 Why do you need a schema registry, and how do you evolve event schemas safely?

🟡 Intermediate | Schemas

💡 Explanation

In event-driven systems, producers and consumers are decoupled and deploy independently — so the **event payload is a contract** between them. A **schema registry** (e.g. Confluent Schema Registry with Avro/Protobuf/JSON Schema) stores and versions those schemas, and enforces compatibility rules so a producer can't publish a change that breaks existing consumers.
Safe evolution follows compatibility modes: **backward compatible** (new consumers can read old events — e.g. adding an optional field with a default), **forward compatible** (old consumers can read new events — e.g. don't remove required fields), or **full** (both). The golden rules: add optional fields, never remove or repurpose existing ones, and never change a field's type in place.

🎯 Scenario

"We renamed `customerId` to `userId` in the OrderPlaced event and three downstream services broke in production." That's the exact failure a schema registry prevents — the registry would reject the incompatible change at publish time. The fix is additive: add `userId`, keep `customerId` populated through a deprecation window, migrate consumers, then remove the old field.

⚠️ Trade-off / Gotcha

Without enforced compatibility, event schemas drift and every change becomes a coordinated multi-team deploy — the very thing microservices are supposed to avoid. A schema registry adds a piece of infrastructure and some process, but it's what keeps an event-driven system from silently breaking as it evolves.

SECTION 07

Observability & Security

You can't fix what you can't see — and you can't trust what you can't verify. The three pillars, distributed tracing, OAuth2/OIDC, zero-trust, and mTLS.



Q40 What are the three pillars of observability, and how do they differ from monitoring?

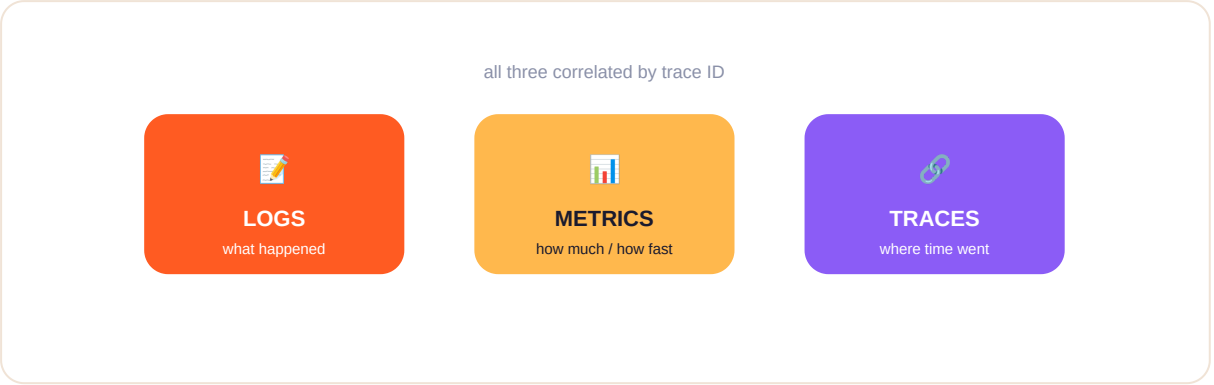
🟢 Beginner | Observability

💡 Explanation

Monitoring tells you *whether* the system is healthy against known questions (CPU, error rate, uptime). **Observability** is the ability to ask *new* questions about what's happening inside — to debug problems you didn't anticipate. In microservices, where one request crosses many services, you need three signal types:

- **Logs** — discrete, timestamped events ("what happened"). Best made structured (JSON) and correlated by trace ID.
- **Metrics** — aggregated numeric measurements over time ("how much / how fast"): request rate, latency percentiles, error counts. Cheap to store, great for dashboards and alerts.
- **Traces** — the end-to-end path of a single request across services ("where the time went / where it failed").

📐 Diagram · the three pillars



⚠️ Trade-off / Gotcha

The pillars are only powerful when **correlated** — a trace ID that appears in logs, metrics exemplars, and the trace itself lets you pivot between them. Three disconnected silos (a logging tool, a metrics tool, a tracing tool that don't share IDs) leave you manually guessing across systems during an incident.

Q41 How does distributed tracing work, and how do you set it up in Spring Boot?

🟡 Intermediate | Tracing

💡 Explanation

Distributed tracing follows one request across every service it touches. A **trace** represents the whole journey; each unit of work within it is a **span** (with parent/child relationships). A trace ID is generated at the entry point and propagated via headers (W3C `traceparent`) to every downstream call, so a tracing backend (Zipkin, Tempo, Jaeger) can stitch the spans into one timeline showing exactly where latency or failures occurred.

In Spring Boot, **Micrometer Tracing** (with the OpenTelemetry or Brave bridge) instruments this automatically — it creates spans for incoming requests, RestClient/WebClient calls, and messaging, and propagates context across them.

⌨️ Code · Micrometer Tracing setup

pom.xml + application.yml

```
<!-- Bridge Micrometer Tracing to OpenTelemetry, export to a backend -->
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>

# application.yml
management:
  tracing:
    sampling:
      probability: 1.0        # 100% in dev; sample down in prod
  otlp:
    tracing:
      endpoint: http://tempo:4318/v1/traces

# spans for RestClient/WebClient/messaging are created automatically
```

⚠️ Trade-off / Gotcha

100% sampling in production is expensive and noisy — sample a small percentage (and/or use tail-based sampling to keep all *error* traces). But sample **consistently** across services: if service A keeps a trace and service B drops it, you get broken, half-complete traces that are worse than none.

Q42 What is zero-trust security, and why does it matter for microservices?

🟡 Intermediate | Security

💡 Explanation

Zero-trust replaces the old "hard shell, soft center" model (trust anything inside the network) with "**never trust, always verify**." Every request — even service-to-service, even inside the cluster — must be authenticated and authorized. The assumption is that the internal network is already compromised, so the network boundary is not a security control.

For microservices this means: authenticate every internal call (JWT, mTLS), authorize by least privilege (a service can only do exactly what it needs), encrypt traffic in transit, and don't rely on "it's behind the firewall" for anything sensitive.

⚠️ Trade-off / Gotcha

The classic breach pattern: an attacker compromises one low-value service, then moves **laterally** because everything inside trusts everything else. Zero-trust contains that blast radius — but it adds latency (token validation, mTLS handshakes) and operational overhead (certificate/secret management). A service mesh is often how teams get zero-trust transport security without rewriting every service.

Q43 How do OAuth2 and OIDC secure microservices? Show a resource server in Spring Security 7.

🔴 Advanced

Spring Security 7

💡 Explanation

OAuth2 is an authorization framework: an *authorization server* (Keycloak, Auth0, Okta, Spring Authorization Server) issues **access tokens** (usually JWTs) that clients present to *resource servers* (your services). **OIDC** (OpenID Connect) adds an identity layer on top — an `id_token` proving *who* the user is. In a microservices setup, the gateway/client obtains a token; each downstream service acts as a **resource server** that validates the token (signature via the auth server's public keys, plus issuer, audience, and expiry) and maps its scopes/roles to authorities.

⌨️ Code · resource server (Spring Security 7, component-based)

SecurityConfig.java

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

  // Spring Security 7: component-based SecurityFilterChain bean
  @Bean
  SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
      .authorizeHttpRequests(auth -> auth
        .requestMatchers("/actuator/health").permitAll()
        .requestMatchers("/orders/**").hasAuthority("SCOPE_orders.read")
        .anyRequest().authenticated())
      // validate incoming JWT access tokens
      .oauth2ResourceServer(oauth -> oauth.jwt(Customizer.withDefaults()));
    return http.build();
  }
}

# application.yml — where to fetch keys to verify the JWT
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://auth.example.com/realms/app
```

⚠️ Trade-off / Gotcha

Always validate **audience** and **issuer**, not just the signature and expiry. A token correctly signed by your auth server but minted for a *different* service (wrong audience) will pass signature checks — accepting it lets one service's token be replayed against another. Scope checks enforce least privilege on top of authentication.

Q44 What is mTLS, and when do you use it over JWT-based auth?

🔴 Advanced | Security

💡 Explanation

Mutual TLS (mTLS) is TLS where *both* sides present certificates — the client verifies the server (normal TLS) *and* the server verifies the client. It proves **service identity at the transport layer** and encrypts traffic, independent of any application-level token. It answers "is this caller really the orders-service?"; JWT answers "what is this user/client allowed to do?" — they operate at different layers and are often used together.

mTLS is usually delivered by a **service mesh** (Istio, Linkerd) via sidecars, so services get automatic, rotating certificates and encrypted service-to-service traffic without app code changes.

🎯 Scenario

"Compliance requires all internal traffic encrypted and every service call cryptographically identified." mTLS via the mesh gives you encryption + service identity transparently; you still layer JWT/OAuth2 on top for *user-level* authorization. Transport identity (mTLS) and user authorization (JWT) solve different halves of the problem.

⚠️ Trade-off / Gotcha

mTLS without automation is an operational nightmare — certificate issuance, distribution, and **rotation** across hundreds of instances. Hand-managed certs expire at 3 a.m. and take down service-to-service traffic. If you adopt mTLS, adopt the automation (mesh or a cert manager) with it; don't roll it by hand.

Q45 How do you secure actuator endpoints and run health probes in microservices?

🟡 Intermediate | Actuator

💡 Explanation

Spring Boot **Actuator** exposes operational endpoints — `/health`, `/metrics`, `/info`, `/env`, and more. In microservices, `/health` drives orchestrator probes: Kubernetes uses **liveness** (restart if dead) and **readiness** (don't route traffic until ready) probes, which Boot exposes as `/health/liveness` and `/health/readiness`. But many actuator endpoints leak sensitive internals, so you must lock them down.

⌨️ Code · expose probes, secure the rest

application.yml

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus   # expose only what's needed
  endpoint:
    health:
      probes:
        enabled: true                 # /health/liveness + /readiness
      show-details: when_authorized       # hide internals from anon

# In SecurityConfig: permit health probes, authenticate the rest
# .requestMatchers("/actuator/health/**").permitAll()
# .requestMatchers("/actuator/**").hasRole("ADMIN")
```

⚠️ Trade-off / Gotcha

Two classic mistakes: (1) exposing `/actuator/env` or `/heapdump` publicly, leaking secrets and config; (2) wiring **liveness** to a deep check that depends on a database — when the DB blips, Kubernetes restarts healthy pods in a crash loop. Liveness should check only that the app itself is alive; dependency health belongs in **readiness**.

🔥 **Master the full backend stack**

Java 8→21, Spring Boot, System Design, Kafka, DSA & more — one TechTribers bundle.

www.techtriber.com

SECTION 08 · 🔥 HOT IN 2026

AI Microservices

The 2026 differentiator. Building intelligent services with Spring AI — RAG-as-a-service, tool calling, the Model Context Protocol (MCP), agentic workflows, and the cost/latency realities of LLMs in a request path.



Q46 What is an "AI microservice," and how does Spring AI fit into a microservice architecture?

🟡 Intermediate | Spring AI

💡 Explanation

An **AI microservice** is an ordinary service whose core capability is powered by a model — summarization, classification, semantic search, a chat assistant, content generation. It exposes a normal API, owns its data, scales independently, and is governed by the same patterns (resilience, observability, security) as any other service. The "intelligence" is just another dependency: a call to an LLM provider.

Spring AI (1.1 GA, with 2.0 targeting Spring Framework 7 / Boot 4) is the Spring abstraction for this. Its central `ChatClient` gives a provider-agnostic, fluent API over OpenAI, Anthropic, Azure OpenAI, local models, etc. — so you write to one interface and swap providers via config, exactly like Spring Data does for databases.

⌨️ Code · a Spring AI ChatClient endpoint

SummaryController.java

```java
@RestController
class SummaryController {

  private final ChatClient chat;

  // ChatClient.Builder is auto-configured from your provider config
  SummaryController(ChatClient.Builder builder) {
    this.chat = builder.build();
  }

  @PostMapping("/summarize")
  String summarize(@RequestBody String text) {
    return chat.prompt()
      .system("You summarize text in 3 concise bullet points.")
      .user(text)
      .call()
      .content();
  }
}
```

⚠️ Trade-off / Gotcha

An LLM call is a **slow, costly, non-deterministic network dependency** — often 1–10+ seconds and priced per token. Treat it with *more* resilience care than a normal call: timeouts, circuit breakers, and ideally async/streaming so you don't block a thread for seconds. Never put an unbounded LLM call directly in a latency-critical synchronous path without these guards.

Q47 What is RAG, and how do you build a "RAG-as-a-service" microservice?

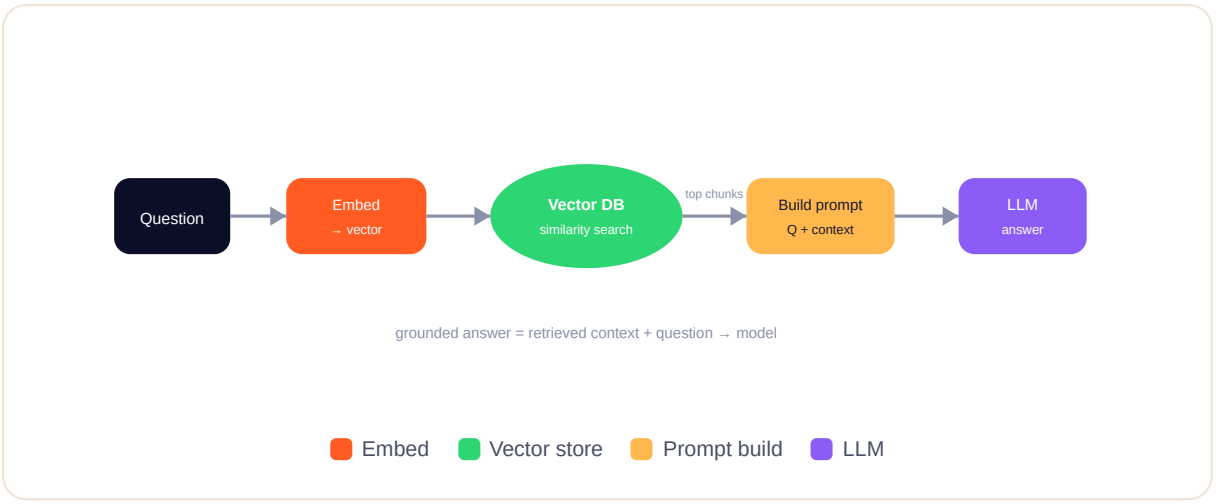
🔴 Advanced | RAG

💡 Explanation

RAG (Retrieval-Augmented Generation) grounds an LLM in *your* data instead of relying only on its training. The flow: documents are chunked and converted to **embeddings** (vectors) stored in a **vector database** (pgvector, Redis, Pinecone, Qdrant). At query time you embed the user's question, retrieve the most semantically similar chunks, and inject them into the prompt as context — so the model answers from your knowledge base and can cite sources. This dramatically reduces hallucination and lets you answer questions about private/current data.

As a microservice, RAG fits cleanly: an **ingestion** path (embed + store documents) and a **query** path (retrieve + generate). Spring AI provides `EmbeddingModel`, `VectorStore`, and a retrieval advisor that wires retrieval into the ChatClient.

📐 Diagram · RAG query flow



⚠️ Trade-off / Gotcha

RAG quality is dominated by **retrieval**, not the model. Bad chunking (too big/small), a weak embedding model, or no re-ranking returns irrelevant context and the LLM confidently answers wrong. "Garbage retrieved → garbage generated." Most RAG effort should go into chunking strategy, embedding quality, and evaluation — not prompt tweaking.

Q48 What is tool/function calling, and how does it let an AI service act on your microservices?

🔴 Advanced | Tool Calling

💡 Explanation

Tool calling (a.k.a. function calling) lets an LLM invoke your code. You describe available functions (name, purpose, parameters) to the model; when a user request needs one, the model returns a structured request to call that function with arguments. Your application executes it — hitting a real microservice — and feeds the result back so the model can respond with grounded, action-backed answers. This turns a chat model from a text generator into something that can *check an order status* or *create a ticket* by calling your existing services.

In Spring AI you register tools as Spring beans/methods annotated as tools; the framework handles exposing them to the model and executing the call-backs.

⌨️ Code · a tool the model can call

OrderTools.java

```
@Component
class OrderTools {

  private final OrdersClient orders;   // calls the real orders service

  // @Tool exposes this method to the LLM with its description
  @Tool(description = "Get the current status of an order by its ID")
  String getOrderStatus(@ToolParam(description="the order id") String id) {
    return orders.getOrder(id).status();
  }
}

// Register tools with the ChatClient — the model calls them when needed
chat.prompt()
   .user("Where is order A123?")
   .tools(new OrderTools(ordersClient))
   .call()
   .content();   // model invokes getOrderStatus("A123") behind the scenes
```

⚠️ Trade-off / Gotcha

Tools give the model **real power** — so guard them. Never expose a destructive or unbounded action (delete data, move money, run arbitrary queries) as a freely-callable tool without authorization checks, input validation, and limits. The model can be prompt-injected into calling a tool maliciously; treat every tool invocation as untrusted input and enforce the same authz as a normal API call.

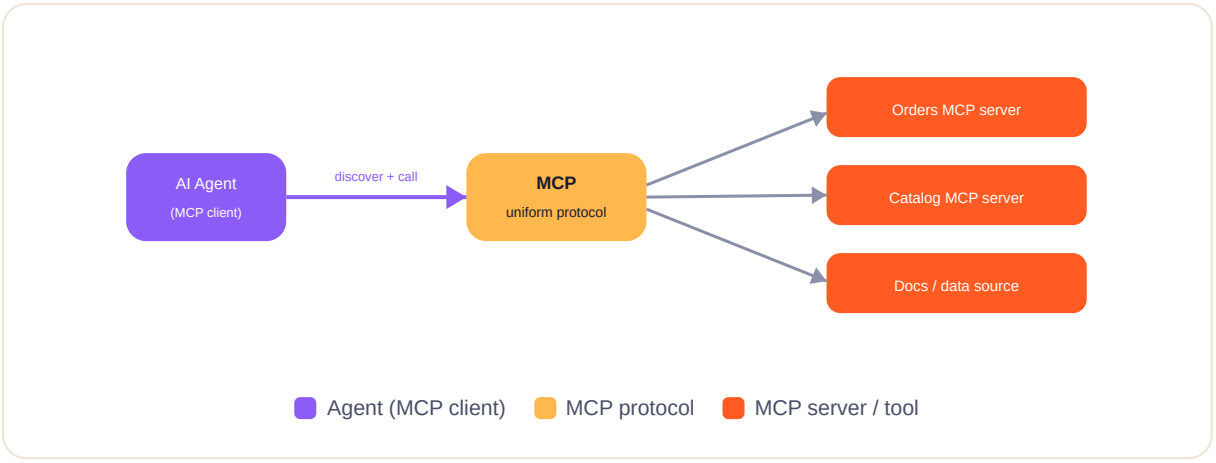
Q49 What is the Model Context Protocol (MCP), and why does it matter for microservices?

🔴 Advanced | MCP

💡 Explanation

The **Model Context Protocol (MCP)** is an open standard for connecting AI models/agents to external tools and data sources through a uniform interface. Instead of hand-wiring each integration, a service exposes its capabilities as an **MCP server**, and any MCP-aware client (an agent, an IDE assistant, another service) can discover and call them. Think of it as a standardized contract for "here are the tools and data I offer the AI" — reducing the N×M integration problem between models and systems. **Spring AI added MCP integration in 1.1**, letting Spring services act as MCP servers or clients.

📐 Diagram · MCP as a uniform tool interface



⚠️ Trade-off / Gotcha

MCP is young and evolving fast — the spec, security model, and Spring AI's support are all maturing through 2026. It's powerful for standardizing agent-to-system access, but treat MCP servers as a **security boundary**: an MCP server hands tools to an AI that may be steered by untrusted input, so apply authn/authz, scoping, and audit logging exactly as you would to a public API. *Verify the current MCP + Spring AI specifics against spring.io before relying on exact APIs.*

Q50 What are agentic workflows, and how do you orchestrate multiple AI agents as microservices?

🔴 Advanced | Agentic

💡 Explanation

An **agent** is an LLM given a goal, a set of tools, and the autonomy to *decide* which tools to call and in what order to accomplish it — looping (reason → act → observe) until done, rather than following a fixed script. **Agentic workflows** coordinate one or more agents to complete multi-step tasks. In a microservices setting this maps naturally: agents become services, tools become calls to other services (often via MCP), and a coordinator (or a lead agent) orchestrates the collaboration — much like the saga orchestration vs choreography distinction from Section 05, now with AI decision-making at the nodes.

Common patterns: **routing** (an agent classifies and dispatches to a specialist), **orchestrator-workers** (a lead agent decomposes a task and delegates to worker agents), and **evaluator-optimizer** (one agent produces, another critiques and refines).

🎯 Scenario

A support assistant receives "I was charged twice for order A123." A lead agent decomposes it: call the *orders* tool to verify the order, the *payments* tool to find duplicate charges, then the *refunds* tool to issue a refund — each a real microservice call via tool/MCP. It composes the result into one answer. The architecture is recognizable microservices; the novelty is the agent *choosing* the steps.

⚠️ Trade-off / Gotcha

Autonomy is the risk. An agent that decides its own actions can loop forever (runaway token cost), take wrong/destructive actions, or be hijacked via prompt injection into misusing tools. Production agentic systems need hard **guardrails**: step/iteration limits, budget caps, human-in-the-loop approval for high-impact actions, scoped tool permissions, and full audit logging. Don't give an autonomous agent unguarded access to money-moving or destructive tools.

Q51 What are the cost, latency, and reliability concerns of putting an LLM in a request path?

🟡 Intermediate | AI Ops

💡 Explanation

An LLM dependency behaves unlike a normal microservice call on three axes:

- **Latency** — responses take seconds, not milliseconds, and vary with output length. Use **streaming** (token-by-token) so users see progress, and async processing to avoid blocking threads.
- **Cost** — priced per token, so cost scales with traffic and prompt/response size. Control it with caching (semantic cache for repeated questions), prompt trimming, smaller models for simple tasks (model routing), and budget caps.
- **Reliability** — providers rate-limit, have outages, and return non-deterministic output. Wrap calls with timeouts, retries with backoff, circuit breakers, and fallbacks (cached answer, smaller model, or a graceful "try again").

⚠️ Trade-off / Gotcha

An **LLM gateway** (a service in front of providers) centralizes these concerns — routing to the cheapest capable model, caching, rate-limit handling, failover across providers, plus cost/usage observability and guardrails. Without it, every team re-implements (inconsistently) retries, caching, and spend tracking, and you have no central view of AI cost or a kill-switch when spend spikes.

READY TO LAND THE OFFER?

Crack your next microservices interview

You've seen the depth — explanations, real Spring Boot 4 & Spring Cloud code, architecture diagrams, and the trade-offs interviewers actually probe for. Get the complete bundle and walk in prepared.

Get the Complete Bundle →

www.techtriber.com

Follow @techtribers for daily backend interview prep

★★★★★

$9.99 · Lifetime Access · Instant Download